

26/8/30 발표 · GITHUB.COM/DAVID1-P

LPG와 RDF

애플리케이션의 그래프 / 공유되는 그래프

그래프 DB에는 크게 두 가지 데이터 모델이 있다 — 그리고 둘 중 하나를 고르는 기준이 있다.
판단의 축은 "내 데이터가 어디까지 이동하는가"입니다.

참고 자료: Tentrис 웨비나 "RDF, Modeled Right — and Queried Fast" (Tobias Rebert, 2026-08)

이 발표에서 다루는 것

	주제	내용
1	두 가지 모델링 방법	LPG와 RDF의 구성 요소와 전제
2	같은 사실, 두 모델	David-Toss-서울을 Cypher와 Turtle로 각각
3	판단을 가르는 네 가지 축	식별자·표준 / 추론·연합 / 경로 질의 / 엣지 속성
4	그래서 무엇을 골라야 하는가	선택 기준과 Enterprise AI 관점
5	한눈에 보는 비교	항목별 대조표

그래프를 정의하고 공유하는 두 방식

둘 다 엔티티와 관계를 그래프로 표현한다. 차이는 그 그래프를 어떻게 정의하고, 어떻게 공유하는가에 있다.

LPG · LABELED PROPERTY GRAPH

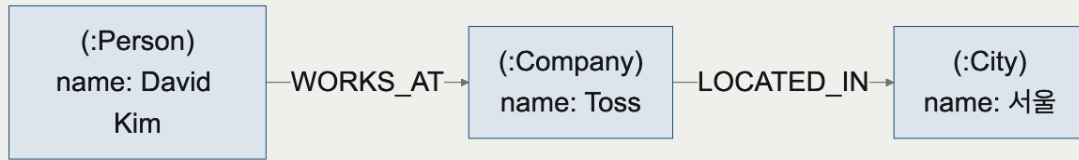
- 노드와 엣지가 각각 임의의 **key-value 프로퍼티**를 가짐
- Cypher, Gremlin, GQL, 벤더별 API로 질의
- 내부 ID는 보통 DB 로컬 — 전역 ID는 애플리케이션이 별도 설계
- 스키마·제약 기능은 제품과 언어에 따라 다름
- 경로 패턴·반복 순회 질의에 대한 뛰어난 사용성

RDF · RESOURCE DESCRIPTION FRAMEWORK

- 모든 사실이 **subject-predicate-object** 트리플
- **SPARQL**로 질의 — 이것도 W3C 표준
- 엔티티에 **전역 IRI** → 조직 경계를 넘어 같은 식별자 공유
- RDFS/OWL·SHACL 등 온톨로지·추론·검증 표준 계층
- 추론·검증의 실제 지원 방식은 스토어마다 다름

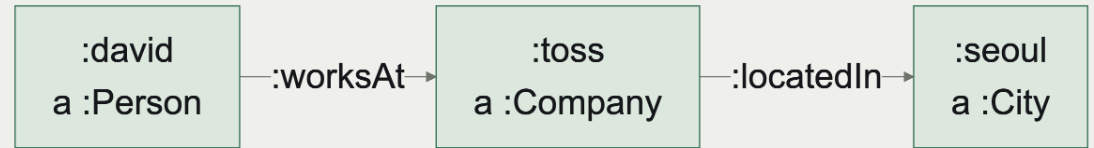
"David는 Toss에서 일하고, Toss는 서울에 있다"

LPG · CYPHER



서울을 `city: "서울"` 프로퍼티로 둘지 `(:City)` 노드로 승격할지는 **모델러의 선택**이다.

RDF · TURTLE



서울은 **IRI로 식별되는 리소스**다. 그 식별자와 `:locatedIn`의 의미를 외부 어휘와 표준 방식으로 공유할 수 있다.

어디서 갈리는가

축	우위	요약
1 · Identity & standards	RDF	전역 IRI와 W3C 표준 계층 vs 애플리케이션이 관리하는 식별자
2 · Reasoning & federation	RDF	추론·연합 질의를 위한 표준 명세 — 지원·설정은 제품별
3 · Path queries	LPG	가변 길이 경로에서 끝점이 아니라 경로 자체 를 반환
4 · Edge properties	LPG	관계에 직접 속성 부여, 중간 노드 불필요

실제 선택은 이 네 축의 **가중치**로 결정된다. 4번은 RDF 1.2 표준화 진행에 따라 다시 평가할 항목이다 — 2026-08 현재 핵심 문서는 Candidate Recommendation / Working Draft 단계.

"10"은 우리 DB 밖에서 아무 의미가 없다

LPG

`id = 10`

RDF

`https://toss.example/company/10`

전역 식별자

네임스페이스 없이 다른 시스템과 통합하면 그쪽 `id 10` 과 충돌한다. IRI는 전역 범위에서 같은 문자열로 비교된다 — 단 지속성과 올바른 지시는 **소유자의 거버넌스 책임**이다.

열린 표준

LPG도 2024년 GQL(ISO/IEC 39075)로 ISO 표준 언어를 갖게 됐다. 남은 차이는 언어가 아니라 **계층**이다 — 어휘·추론(RDFS/OWL), 검증(SHACL), 연합 질의(SERVICE).

온톨로지 한 줄이 답을 바꾼다

```
:Company rdfs:subClassOf :Organization .
```

이 한 줄이면 모든 `:Organization` 을 물었을 때 Toss가 결과에 나온다. Toss는 `:Company` 라고만 적어 뒀는데도.

추론

RDFS·OWL은 단언한 사실에서 **새 사실을 도출하는 표준 의미론**을 준다. 다만 실제로 답이 나오려면 entailment·materialization·query rewriting이 **켜져 있어야** 한다.

연합 질의

SPARQL SERVICE 는 하나의 쿼리를 **여러 엔드포인트에 걸쳐** 실행하는 표준 문법이다. 실행은 원격 endpoint의 가용성·권한·성능에 의존한다.

LPG에는 이 둘에 해당하는 공통 표준이 없다. 못 한다는 뜻이 아니라, 제품 기능이나 애플리케이션에서 **매번 다시 만들어야** 한다는 뜻이다.

경로가 무엇이었는지 말해줄 수 있는가

LPG · CYPHER

```
MATCH p = shortestPath(  
  (a:Person {name:"David"})  
  -[:REPORTS_TO*1..5]->  
  (b:Person {name:"Dana"})  
)  
RETURN p
```

경로 자체를 반환한다. 모든 홑을 순서대로, `shortestPath()` 내장, 길이 제한, 경로상 엣지 속성 필터까지.

RDF · SPARQL 1.1

```
SELECT ?manager WHERE {  
  :david :reportsTo+ ?manager .  
}
```

끝점만 반환한다. 임의 깊이까지 한 줄로 답하지만 중간 노드 바인딩·경로 반환·엣지 속성 제약은 못 하고, 표준 `shortest-path`도 없다.

질문이 **경로**에 관한 것이면 LPG, **도달 가능성**에 관한 것이면 SPARQL 한 줄로 끝난다.

속성이 관계에 붙을 때

LPG • 네이티브

```
(david)-[:WORKS_AT {  
  since: 2028,  
  role: "Engineer"  
}]->(toss)
```

"2028년부터 Engineer로" — since 와 role 은 David의 것도, Toss의 것도 아니다. 관계의 속성이다.

속성이 엣지 위에 그대로 앉는다. 간결하고, 즉시 질의 가능하고, 추가 모델링이 없다. LPG의 가장 명확한 사용성 우위.

RDF의 두 가지 선택지

OPTION A · 중간 노드 (PLAIN RDF)

```
:employment1 a :Employment ;  
  :employee :david ;  
  :employer :toss ;  
  :since    2028 ;  
  :role     "Engineer" .
```

장황하지만 **표준 RDF만으로** 동작하고, 어느 스토어에서나 질의된다.

OPTION B · RDF 1.2 ANNOTATION (표준화 중)

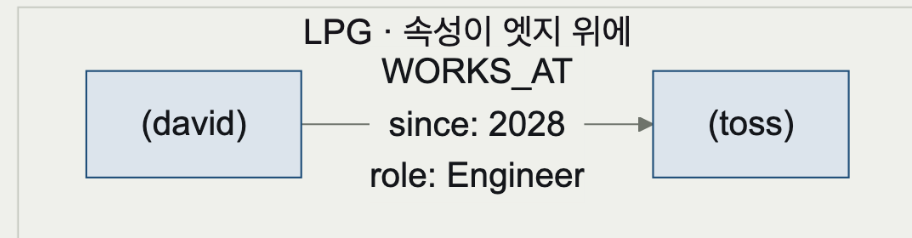
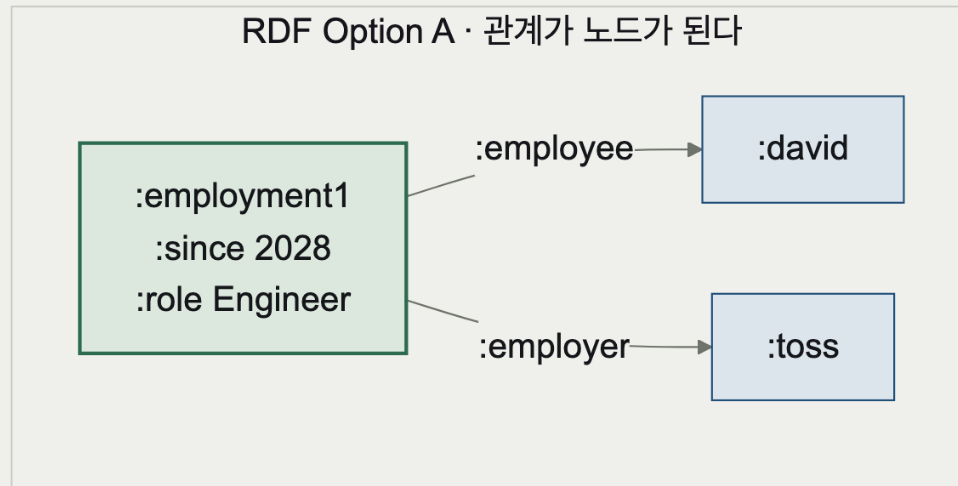
```
VERSION "1.2"  
:david :worksAt :toss  
  { | :since 2028 ; :role "Engineer" | } .
```

발생 자체에 ID가 필요하면 **reifier**를 명시한다:

```
:david :worksAt :toss  
  ~ :employment1 { | :since 2028 | } .
```

주의: { | ... | } 는 엣지에 속성을 붙이는 기능이 **아니라** asserted triple의 암시적 reifier에 메타데이터를 기술하는 축약이다. Turtle/SPARQL 1.2는 Working Draft — 구현 지원을 확인하고 쓸 것.

그 관계는 자기 자신의 삶을 갖는가



간결함은 **LPG** 쪽이 낫다. RDF의 중간 노드는 더 장황하다 — 하지만 그 대가로 `employment`가 **일급 엔티티**가 된다. SHACL로 검증하고, 계약서에 링크하고, 추론할 수 있다.

데이터가 어디까지 이동하는가

LPG를 유지할 때

- 그래프가 하나의 애플리케이션을 벗어나지 않고, 바깥으로 링크할 일도 없다
- 팀이 이미 Cypher나 Gremlin에 능숙하다
- 엣지가 자체 속성을 갖거나, 질문이 경로에 관한 것이다 (축 3 & 4)

RDF를 선택할 때

- 데이터가 팀·시스템·조직의 경계를 넘는다
- 거버넌스가 필요하다 — 출처(provenance), 검증, 감사
- 자사 지식 위에 **Enterprise AI**를 올린다

양자택일이 아닌 경우도 많다. 하위 계층에 RDF로 표준화·추론을 깔고, 그 위에 LPG를 얹어 경로 질의를 돌리는 조합도 쓴다 — **동기화 비용까지 함께** 계산할 것.

어시스턴트는 정당화할 수 있는 만큼만 신뢰받는다

전역 IRI

답이 가리키는 엔티티를 **시스템 간에 일관되게** 식별한다.

온톨로지

추론 엔진이나 query rewriting을 통해 **명시되지 않은 사실**도 검색할 수 있다.

연합 질의

외부 지식이 **복사 없이** 쿼리에 합류한다.

정직한 단서: IRI만으로 출처가 생기지는 않는다. provenance에는 named graph·PROV-O·reification 등 별도 모델링이 필요하다. LPG로도 RAG 파이프라인을 훌륭히 구축할 수 있고, 필요한 신원·스키마·추론 계층을 제품 기능이나 애플리케이션에서 구성할 수 있다.

표준과 식별자

항목	RDF / SPARQL	LPG (CYPHER / GREMLIN)
질의 언어	SPARQL — W3C 권고 (2013)	Cypher / openCypher / GQL — ISO/IEC 39075:2024
식별자	전역 IRI가 데이터 모델의 일부	내부 ID는 로컬, 전역 ID는 애플리케이션이 설계
스키마 & 온톨로지	RDFS / OWL / SHACL	GQL graph type·제품별 제약, 공통 온톨로지 계층은 없음
추론	RDFS/OWL 표준 의미론 — 구현·설정 필요	공통 표준 아님 — 제품/앱 레벨에서 구현
연합 질의	SPARQL 1.1에 내장	공통 표준 아님 — 제품·애플리케이션별 구현

질의와 적합성

항목	RDF / SPARQL	LPG (CYPHER / GREMLIN)
엣지 속성	중간 노드, 또는 RDF 1.2 annotation (<i>표준화 중</i>)	엣지에 네이티브 key-value
경로 질의	Property path — 끝점만	가변 길이, 경로를 반환
적합한 곳	시스템을 넘는, 링크되고 통제되는 데이터	자기완결적 애플리케이션 그래프

마지막 줄이 결론이다 — 데이터가 조직을 넘어가면 RDF, 애플리케이션 안에서 끝나면 LPG.